

End-of-Quarter Reflection

HCDE 530 | Rachit Singhi

I came in as a UX researcher and designer who could read a chart but could not really build one, and who treated a dataset as a finished object handed to me rather than something I had to interrogate. Looking back through my repo now, the commit history, the weekly notebooks, the two major projects, I can see the actual shape of what changed. It was not that I learned to write a large amount of code. It was that I learned to be the person who decides whether the code, and the data behind it, can be trusted, and who can explain that decision to someone else.

This reflection makes the case for three of the eight domains: C5, C7, and C8. I picked them because together they trace the real arc of my quarter. C5 is learning to ask a dataset a precise question. C7 is learning to not believe an answer just because it looks finished. C8 is learning to ship something a stranger can actually use without me standing next to them. For each one I name the domain, point to specific files and findings you can open and verify in the repo, explain what being able to do it shows about my capability, and say honestly what I learned in the process. I left C2 (Code Literacy and Documentation) out on purpose, even though there is real evidence for it, because my own `Week 3/week3.md` admits my early commit messages such as "Fixed 2 bugs" were too vague, and the git log proves it. I would rather make a strong case for three domains than a partial case for four.

C5 - Data Analysis with Pandas

Where to look: `Week 5/MP1/a5_analysis.ipynb`, `MP1/mp1.md`, the C5 write-up in `Week 5/MP1/week5.md`, and the operations table in `README.md`.

For MP1 I worked with the HYG v4.2 star catalog, which holds about 119,626 real stars, each with position (`ra`, `dec`), distance, apparent and absolute magnitude, a B-V color index (`ci`), spectral class (`spect`), luminosity, and constellation. I used pandas to answer three connected questions: can stars be grouped into constellation-like clusters by position, do a few bright stars anchor each cluster, and do different clusters hold physically different kinds of stars. I used all five operations we covered in class, `head`, `info`, `isnull`, filtering, and `value_counts`, plus `groupby` repeatedly, but the part I am genuinely proud of is not that I called them. It is that I can explain why I reached for each specific one to answer each specific sub-question, which is the difference between running pandas and doing analysis with it.

The clearest example is the filtering step, `df[df['mag'] < 6.5]`, which reduced the catalog to roughly 9,000 naked-eye-visible stars before any clustering happened. That looks like a one-line throwaway, but it was the decision the entire project rested on. Magnitude runs backward, lower numbers are brighter, and 6.5 is roughly the limit of what a human eye can see in a dark sky. If I had run K-Means on all 119,626 rows, the clusters would have been dominated by stars too faint to ever see with the naked eye, and a "constellation generator" built on invisible stars would be meaningless in practice. So choosing where to put that filter was really choosing what question I was answering. I learned that the most consequential line of an analysis is often not the modeling step, it is the quiet filter near the top that defines the population, and that a reader deserves to know why the threshold sits where it does.

A second moment came from `df.isnull().sum()`, which showed me that roughly 3,000 stars were missing spectral class data. I would never have noticed that from `df.head()` alone, because the first five rows looked perfectly clean. That single count changed how I scoped the spectral composition question: instead of quietly running an analysis over thousands of blanks and reporting a confident but corrupted result, I worked on a filtered subset and said so in the notebook. What I learned here is that missing data is not an edge case to handle at the end, it is information about the dataset itself, and checking for it early is how you avoid publishing a clean-looking number that is actually built on holes.

The real finding came from `groupby('cluster')` on the magnitude column. Across nearly every cluster, the brightness distribution had the same right-skewed shape: a dense band of dim filler stars near magnitude 5 to 6, and a short tail reaching two to four genuinely bright anchor stars. The point was not that one cluster behaved this way, it was that almost all of them did. That consistency is the actual result, because it suggests a structural property of how the night sky is organized rather than a coincidence in one lucky patch. Being able to see that required grouping and then comparing across groups, not just describing one. I learned that the interesting answer is frequently a pattern that holds across many groups, and that aggregation is how you make that pattern visible instead of guessing at it.

One more piece of evidence is small but it captures the whole mindset. Running `describe()` before writing any visualization code caught a 100,000-parsec sentinel distance value sitting in the data. Left alone, that placeholder would have thrown an impossible outlier point into a later 3D chart and made the chart look broken for a reason that is very hard to diagnose after the fact, when you are staring at a finished plot wondering why it is wrong. Catching it at the profiling stage cost me one cell. Catching it after the chart shipped would have cost me an afternoon.

What this demonstrates and what I learned: I can take a large, real, imperfect dataset and turn a vague curiosity into a specific, testable question, then choose the pandas operation that answers that exact question rather than running everything and hoping something stands out. More than the syntax, I learned a sequence I now treat as non-negotiable: profile the data before you analyze it, define your population on purpose, check for what is missing, and look for patterns that hold across groups rather than anecdotes from one. That sequence is the difference between a number I would defend in a meeting and a number I just produced.

C7 - Critical Evaluation and Professional Judgment

Where to look: Week 6/MP1/week6.md , the C7 section of MP1/mp1.md , and the three documented moments in MP2/mp2.md .

This is the domain where I changed the most, and it shows up in both major projects, which is exactly why I picked it. I do not have one example, I have several, and each one forced an actual decision rather than a vague feeling that something might be off.

The first was the MP1 dataset itself. I began with three CSV files I had pulled together, a large N-body simulation, a sky survey, and a short list of named stars, and I asked Claude to merge them into one unified star catalog. The merged file came back looking perfect: every row populated, no errors, a tidy table. But Claude flagged that the three files shared no common key, which meant the physical properties had been randomly paired to positions they had no real relationship to. A hot blue star could end up sitting at a cool red dwarf's coordinates, which is physically impossible. The output looked complete and was substantively wrong. Instead of accepting it because it was tidy, I treated that flag as a real problem. I looked at the recommended alternative, the HYG database, and before trusting it I spot-checked it against published values for two stars I could verify by hand, Sirius and Betelgeuse, to confirm the columns matched reality. Only then did I throw the synthetic file away and rebuild on HYG. What I learned is that a perfectly formatted table with invented relationships inside it is more dangerous than no table at all, because it fails silently and confidently, and the only defense is to check it against a ground truth I trust.

The second moment was in MP1b. Cursor built me a 3D version of my cat constellation exactly as I asked, using the stars' real distances in Cartesian space, and it ran with no errors at all. But when I looked at it, the cat was unrecognizable. The nine anchor stars sit between 49 and 411 parsecs from Earth, so connecting them in true 3D stretched the outline into a diagonal slab along the line of sight. The code worked and the output was useless. I made two judgment calls.

First, I went back to the boring 2D sky projection, because that is genuinely the correct representation for a constellation, which is a line-of-sight pattern from where we stand, not a shape floating in space. Second, I kept the failed 3D version in the notebook as Chart 1b on purpose, sitting next to the version that works, because the failure is itself the finding. I learned that the right fix is sometimes the less impressive one, and that showing a failure honestly teaches the reader more than hiding it. A third, related catch in the same project was noticing that a Plotly drag-to-draw widget ran fine on my machine but would not render on GitHub, where the notebook would actually be graded, so I rewrote it into a pattern that works in any viewer. The test I used was simple: would a grader opening this on GitHub see working output or a broken widget.

MP2 pushed this further, and the honest write-up in `MP2/mp2.md` documents three failures I had to catch and redirect, each of which changed the architecture of a feature. First, I asked Groq to return arrays of x,y coordinates that traced a shape like a cat face, and the output was random lines with no resemblance to anything, even after I added few-shot examples. The root cause, once I named it, was clear: a language model is trained on text, not geometry, so it has no reliable spatial reasoning about coordinate systems. The fix was to stop asking AI to do geometry and use it only for the text matching it is genuinely good at, sending the user's words to the Iconify icon library and parsing the real SVG path that comes back. Second, I tried Google's Quick Draw dataset and hit a CORS wall, then routed around it through an edge function only to find the latency was an unreliable two to eight seconds because the source files are enormous. I judged that a demo that works once is not a feature a user can depend on, and switched to an approach under 500 milliseconds. Third, the MediaPipe finger-drawing feature placed a new vertex on every tiny hand tremor, so I built a calibration step that measures the user's palm size in pixels and scales the movement threshold to how far their hand is from the camera, instead of trusting a single fixed number the tool had picked.

What this demonstrates and what I learned: I can tell the difference between output that looks correct and output that is correct, and I can name the concrete test I would use to decide. That is the part of working with AI that a stakeholder actually pays a person for. What I learned, across all of these, is that critical evaluation is not skepticism for its own sake and it is not catching typos. It is knowing what question I am really trying to answer before I accept an answer, separating what a tool is good at from what it merely sounds confident about, and always verifying against either a ground truth or the real deployment context. I learned to ask three questions out of habit now: is this the right tool for this job, is it reliable enough for a real user, and would I put my name on it in front of a client without checking it first.

C8 - Building and Deploying a Complete Tool

Where to look: the live site at <https://twinkle.ltd>, `MP2/mp2.md`, `MP2/reflection.md`, the original declaration in `Week 8/MP2a.md`, the data pipeline in `Week 8/prepare_stars.py`, and the components under `MP2/src/`.

Twinkle is my MP2. It is deployed on a custom domain and anyone with a browser can use it, which is the part that makes it "complete" rather than "working on my machine." The full flow runs end to end: a person draws a shape, each vertex snaps to the nearest real star from the HYG catalog, a Groq-powered AI gives the constellation a name, and they can view it in 2D or 3D and download it as a PNG. On a phone, a rear-camera mode overlays the result on the real night sky. The thing I want to argue is not that it looks nice, it is that it is built as a chain of honest layers, and that I can point to each layer and explain the decision behind it.

The data does not come from the app, it comes from a separate Python script, `Week 8/prepare_stars.py`, and the details there are verifiable. It filters the raw HYG catalog to magnitude under 6.5 and distance greater than 0.01 parsecs, which is how the roughly 119,000-star catalog becomes the about 8,835 stars the site actually loads. It converts right ascension from hours to degrees with `ra_deg = ra_h * 15.0`, because HYG stores RA in hours but the web renderer needs degrees, and getting that one multiplication wrong would silently rotate the entire sky. It precomputes Cartesian `x`, `y`, `z` coordinates with a `to_xyz()` function so the browser does not have to do trigonometry for thousands of stars at render time. And it fills missing values with safe defaults through small `_float` and `_int` helpers, so a blank color index becomes 0.0 (white) and a blank spectral class becomes "Unknown" rather than crashing the front end. That preprocessing discipline is the same lesson I first practiced in Week 4, where `export_pexels_to_csv.py` loaded an API key from a `.env` file with `os.environ.get` and dug two levels into the JSON response to pull nested image URLs out of the `src` object. The Week 4 habit of keeping keys out of the code carried directly into MP2, where the Groq API key lives in a Cloudflare Workers edge function rather than the browser, because anything in client code can be read by anyone in their developer tools.

The part I think matters most for this domain is what I deliberately cut. My original declaration is in `Week 8/MP2a.md`, and `MP2/reflection.md` documents the scope decisions I made from it: Supabase persistence was cut entirely, and freehand drawing was replaced with straight-line polyline segments — both verifiable in the repo. The declaration had included an optional Supabase `used_names` table to check new constellation names for global uniqueness; I cut even that, and shipped with no database at all. Adding the Supabase integration, plus the row-level security and authentication it would have pulled in once names were stored, was real

work that the core experience, create and download, does not actually need. Cutting a feature I had already named in my own declaration was uncomfortable, and `MP2/reflection.md` says so plainly, but it clarified what the product really is: a creative instrument, not a social platform. I learned that scope is not a list of everything possible, it is a decision about what the product is for, and that shipping the smaller honest version is a stronger move than missing the deadline chasing the bigger one.

I also know exactly where the seam is that I did not finish, and I think naming it is part of shipping. The AR sky view uses the phone's compass heading from `deviceorientationabsolute` but does not use GPS, so the overlay is directionally close but not astronomically exact, because the correct mapping from compass heading to a region of sky depends on where on Earth you are standing and the local time. The fix, which I describe in both `mp2.md` and `reflection.md`, is a one-time `navigator.geolocation` permission and a local sidereal time calculation. I did not pretend the seam was not there, and I can tell you precisely what I would do with another week.

What this demonstrates and what I learned: I can scope a real project for a real human-centered use case, make the architecture decisions that keep it working for a stranger and not just on my laptop, secure the parts that need securing, and be honest about the trade-offs. What I learned is that "complete" does not mean every feature from the original plan. It means every step of the user journey works reliably, from a Python preprocessing script, to a static CSV served by the front end, to an API key hidden behind an edge function, to a custom domain a person can open on their phone. I also learned that each layer only works if the one below it is honest about its assumptions, and that knowing exactly where your seams are is what separates a finished project from a project that merely looks finished.

One thing I learned that was not in the objectives

The syllabus taught me tools. The thing I did not expect to learn was how to manage one.

By the end of the quarter I stopped treating AI like an oracle that hands me finished answers and started treating it like a fast, confident junior collaborator that I am responsible for supervising. That sounds like a small reframe, but it changed how I work day to day. I now assume that anything a tool gives me is a draft, and that my actual job is to find the one place it went quietly wrong, because the quiet wrongness is the expensive kind. The merged star CSV

taught me this first. It was the moment I really understood that "looks complete" and "is correct" are two different properties, and that only one of them is the tool's responsibility while the other one is mine.

The test I built out of that habit is simple: find one value in the output I can verify independently, check it, and if it's right, trust the pattern. If it's wrong, find out why before going further. That test takes thirty seconds and has saved me hours three separate times this quarter. Before I trust a feature I imagine a real person using it in a worse situation than mine — a shaky hand far from the camera, a phone with no location permission, a notebook opened on GitHub instead of my laptop.

The second thing I learned was more uncomfortable than the AI one because it meant choosing the less impressive option on purpose. When I built the 3D constellation view in MP1b, the result was technically correct and experientially useless. The cat face dissolved when rotated because the anchor stars sit at real distances between 49 and 411 parsecs and the outline only exists when viewed from exactly Earth's angle. The right fix was the 2D sky projection, which is simpler, less dramatic, and actually correct for what a constellation is. I kept the 3D version in the notebook as Chart 1b precisely because the failure is the finding, and hiding it would have been dishonest about what the data shows. What I learned from that is a principle I think applies directly to design work: the right representation for a question is not always the most sophisticated one, and choosing a simpler tool because it fits the problem better is a stronger judgment than choosing a complex one because it demonstrates more skill. In HCD terms, the question "what does this person actually need to understand" should come before "what is the most impressive thing I can build." I knew that as a principle from research. I had not really practiced it as a builder until this quarter.

The third thing I learned was about a category of failure I did not have a name for before this quarter. Every error I had encountered before produced a signal — a crash, a traceback, a broken chart, something visually wrong. What I did not know how to look for was the failure that produces nothing. The three-CSV merge completed without a single warning and returned a perfectly formatted table. The 100,000-parsec distance values in the HYG catalog sat in the dist column looking like normal numbers. The synthetic file's physical properties were correctly typed, correctly formatted, and completely fabricated in terms of which star they actually described. None of these threw an error because none of them were errors in a technical sense. The data was structured correctly. It was just wrong. What I learned to do, slowly and mostly by getting burned, was to build a verification step into my workflow before I trusted any output I had not personally traced from source to result. For the HYG catalog that meant checking Sirius and Betelgeuse against published values before building anything on top of the data. For the dist column it meant running `df.describe()` before writing a single

visualization and asking whether the maximum value made physical sense. For the merged CSV it meant taking the flag seriously rather than dismissing it because the table looked tidy. The habit that came out of all of this is one sentence I now ask before moving forward: could this be wrong in a way that would not tell me. If the answer is yes, I find the check before I trust the output. That question did not come from a slide. It came from several hours of debugging analyses that produced confident wrong answers, from rubric feedback across the quarter pushing me to be specific about what I actually verified rather than what I assumed, and from eventually realizing that the most dangerous kind of error is the one that looks like it is done.

I think this is the most useful thing I am leaving the quarter with, because it transfers far beyond this course. It is not really about stars, or pandas, or Cloudflare Workers. It is a way of working where I stay accountable for the output no matter how it was produced, and where I can explain to a colleague or a stakeholder exactly what I checked, what I chose to trust, and why. For a researcher and designer who is now also someone who builds things, that judgment is worth more than any single tool I picked up, and it is the part of this quarter I expect to still be using years from now.